

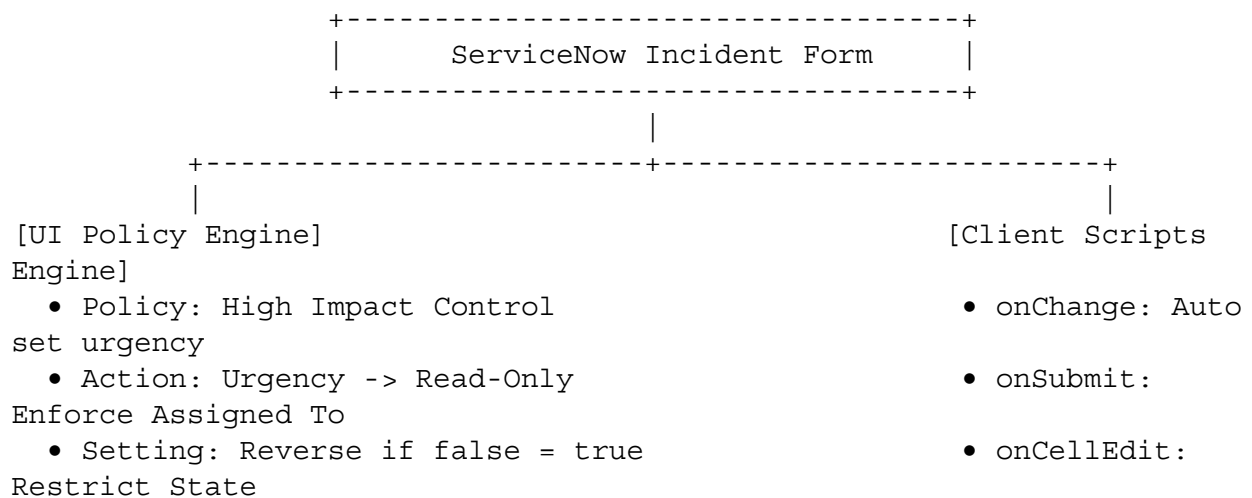
## Project Documentation Phase

This documentation provides a comprehensive operational overview, architectural summary, maintenance guidelines, and user guide for the **Implement Client Script & UI Policy (Incident)** project in ServiceNow.

### 1. Executive Overview

- **Project Name:** Implement Client Script & UI Policy (Incident)
- **Target Application:** Incident Management (incident table)
- **Core Objective:** Ensure data integrity and operational consistency on Incident records by applying dynamic, client-side validation, automated field population, read-only field policies, and list-edit controls.
- **Implementation Summary:**
  - **1 UI Policy:** Configured to set the Urgency field to read-only when Impact is set to High (1).
  - **3 Client Scripts:** Cover onChange (auto-population and user notification), onSubmit (save validation for assigned\_to), and onCellEdit (restricting state modifications via list view).

### 2. System Architecture & Component Inventory



#### A. UI Policy Configuration

| Parameter        | Configuration Value      |
|------------------|--------------------------|
| Policy Name      | High Impact Control      |
| Table            | Incident (incident)      |
| Conditions       | [Impact] [is] [1 - High] |
| Active           | true                     |
| Reverse if false | true                     |

|                         |                                                          |
|-------------------------|----------------------------------------------------------|
| Parameter               | Configuration Value                                      |
| <b>UI Policy Action</b> | <b>Field:</b> Urgency (urgency)   <b>Read-only:</b> true |

## B. Client Scripts Technical Inventory

| Script Name                                | Type       | Target Field / Event  | Execution Logic                                                                                     |
|--------------------------------------------|------------|-----------------------|-----------------------------------------------------------------------------------------------------|
| <b>Auto set urgency for high impact</b>    | onChange   | impact                | When impact is set to '1', urgency is set to '1' and an info banner displays.                       |
| <b>Prevent save if Assigned To missing</b> | onSubmit   | Form Submission Event | If impact == '1' and assigned_to is blank, displays an error box and cancels submit (return false). |
| <b>Prevent state change via list edit</b>  | onCellEdit | state                 | Alerts user to use the form and cancels list update (callback(false)).                              |

## 3. Source Code Repository

### Script 1: onChange Client Script

- **Name:** Auto set urgency for high impact
- **Table:** Incident (incident)
- **Type:** onChange | **Field:** impact

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

### Script 2: onSubmit Client Script

- **Name:** Prevent save if Assigned To missing
- **Table:** Incident (incident)
- **Type:** onSubmit

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {
        g_form.showErrorMessage(
            'assigned_to',
```

```

        'Assigned To is mandatory for High impact incidents.'
    );
    return false;
}
return true;
}

```

### Script 3: onCellEdit Client Script

- **Name:** Prevent state change via list edit
- **Table:** Incident (incident)
- **Type:** onCellEdit | **Field:** state

```

function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the
Incident.');
```

```

    callback(false);
}

```

## 4. End-User Guide & Operational Rules

1. **Creating/Updating High-Impact Incidents:**
  - Setting **Impact** to **1 - High** automatically updates **Urgency** to **1 - High** and locks the field from manual modifications.
  - An informational banner will appear confirming the urgency update.
2. **Submitting High-Impact Incidents:**
  - If **Impact** is set to **1 - High**, you must fill out the **Assigned To** field. Attempting to submit without an assignee will trigger an inline error message and prevent saving.
3. **Updating Incident State in List Views:**
  - Direct inline editing of the **State** field from list views is restricted. Users must open the Incident form to perform state updates.
4. **Reverting Impact Values:**
  - If **Impact** is changed away from **1 - High** to Medium or Low, field locking on **Urgency** automatically releases.

## 5. Maintenance & Best Practices

- **Low-Code First Principle:** For future extensions requiring field behavior changes (visibility, mandatory, read-only), prefer creating additional **UI Policy Actions** rather than writing custom g\_form scripts to reduce technical debt.
- **Server-Side Backing:** Client-side validations (onSubmit / onCellEdit) operate in the browser tier. If API integrations or background imports update Incidents, complementary server-side controls (such as Data Lookup Rules or Business Rules) should be maintained for end-to-end data integrity.